CLOCK.MD

The Ultimate Guide to Temporal Awareness in AI Agents

Why your agent thinks every moment is now — and how to fix it without teaching it philosophy.

- **The problem:** time blindness breaks tool freshness, deadlines, and continuity.
- **The research:** timestamps alone don’t make models behave; per-turn time cues matter.
- **The solution:** a layered protocol (CLOCK.md) plus a deterministic temporal controller.

The Ultimate Guide to Temporal Awareness in AI Agents

Why your LLM thinks every moment is *right now* --- and what to do about it

Here is a fact that should unsettle you more than it probably does: the large language model you entrusted with your calendar, your customer support queue, and possibly your retirement portfolio has no idea what time it is. Not in the vague, jet-lagged way a human might lose track after a transatlantic flight. In the profound, existential way a rock does not know what time it is. The rock, at least, has the decency not to schedule your meetings.

This guide covers three things:

- **The problem** --- LLMs have no sense of time passing, which leads to stale data, missed deadlines, and the conversational equivalent of waking someone mid-surgery and asking them to keep cutting.
- **The research** --- Two papers dissect exactly *how* temporal blindness manifests, and (more usefully) what fails to fix it.
- **The fix** --- A layered protocol called CLOCK.md that gives agents genuine temporal awareness without requiring a degree in distributed systems or a personal relationship with your deployment engineer.

Table of Contents

---

1. The Four Failure Modes

It would be convenient if temporal blindness were a single, well-defined bug --- the kind you could fix with a patch and a firm handshake. It is not. It is four bugs wearing a trenchcoat, each pretending to be the others.

Failure Mode 1: Temporal state is missing entirely. The agent has been given no time context whatsoever. Ask it what timezone it is operating in and it will stare at you with the blank confidence of a golden retriever who has been asked to file taxes. There is no timestamp in the prompt, no timezone anchor, nothing. The model exists in a featureless temporal void, which --- to be fair --- sounds peaceful, but makes for a terrible scheduling assistant.

Failure Mode 2: Temporal state is present but ignored. This is the more insidious variant. Timestamps exist. They are right there in the prompt, clearly formatted, impossible to miss. The agent misses them. It re-calls a tool ten seconds after receiving identical data, like someone who checks their phone, puts it down, then immediately picks it up again to check

the same notification. Tokens are burned, latency accumulates, and the universe grows incrementally more tired.

Failure Mode 3: Continuous time goes untracked. The model has no concept of a ticking clock, a shrinking budget, or an approaching deadline. Tell it "you have thirty seconds left" and it will nod agreeably and then embark on a four-paragraph exploration of the topic's historical context. It is not being defiant. It simply does not understand that thirty seconds is a quantity that decreases.

Failure Mode 4: Human expectation misalignment. This one is subtler. The agent may handle timestamps competently during a session but fails at the *social* contract of time. A user returns after three months of silence. The agent picks up mid-sentence, as though the user had merely paused to sneeze. No "welcome back." No acknowledgment that seasons have changed, governments have fallen, and the user's toddler has learned to walk. Just a cheerful continuation of a conversation about pasta recipes from last spring.

The critical insight --- and this is the part worth underlining, possibly in red --- is that these are *different problems requiring different solutions*. A single "just add timestamps" approach addresses Failure Mode 1, partially, on a good day, if the wind is favorable. The other three require entirely different machinery.

---

2. What the Research Says

Two recent papers had the good sense to put temporal blindness under a proper microscope rather than just complaining about it on the internet, which is the approach most of us had been taking.

2.1 --- "Your LLM Agents are Temporally Blind"

> *arXiv:2510.23853*

The central finding is both unsurprising and dismaying: **tool-use decisions do not reflect elapsed time.** Agents gleefully re-call tools when the data is still warm from its last retrieval, burning tokens and latency for the privilege of learning what they already know. Conversely, when data has gone genuinely stale --- when the stock price has moved, the weather has changed, or the user's meeting has been rescheduled --- the agent serenely serves its cached answer like a waiter bringing yesterday's soup with complete confidence.

The authors built **TicToc**, a benchmark with low, medium, and high time-sensitivity scenarios, and ran a battery of models through it to see which could appropriately modulate their tool-call frequency based on how much time had passed. The results were instructive in the way a bridge collapse is instructive.

What the researchers tried that did not work:

Adding timestamps to the system prompt produced marginal improvement at best --- the model equivalent of writing "REMEMBER: TIME EXISTS" on a sticky note and hoping for the best. Minimal reminder prompts like "be aware of time" had essentially no effect, which will surprise no one who has ever tried to make a cat aware of anything. Few-shot rules helped some models inconsistently, in the way that shaking a vending machine sometimes produces a snack and sometimes produces a bruise.

What actually worked:

DPO (Direct Preference Optimization) and post-training on time-sensitive examples demonstrated that temporal behavior is *learnable* --- the model can develop temporal intuition if you train it in, rather than hoping it spontaneously evolves one. Additionally, a **deterministic controller** sitting outside the LLM that manages tool-call freshness via TTLs and sensitivity classes proved effective. This is the unglamorous solution: don't ask the LLM to be your time-policy engine. Build a boring, reliable layer that decides *when* tools need refreshing, and let the LLM do what it is actually good at, which is reasoning about the results.

2.2 --- "Real-Time Deadlines Reveal Temporal Awareness Failures"

> *arXiv:2601.13206*

This paper investigates what happens when agents face **real-time deadlines** --- the kind humans navigate constantly and mostly without combusting. "You have five minutes to finish this report." "The client call starts in ten." "The building is on fire and we need the quarterly numbers before the ceiling collapses."

The findings are bracing. Performance is **poor** unless remaining time is explicitly supplied on every single turn. The model will not infer urgency from context. You must hand-deliver the information, turn by turn, like feeding coins into a parking meter.

More interestingly, a **qualitative urgency prefix** --- something like "Running low on time" --- can actually *outperform* a raw numeric countdown. Telling the model "47 seconds remain" is less effective than telling it "this is getting urgent." There is something almost poetic about the fact that an artificial intelligence responds better to vibes than to numbers, though "poetic" may not be the word your project manager uses when you explain this to them.

The takeaway: remaining time is a **first-class state variable**. Inject it per turn, at the point of decision. And consider that sometimes a well-placed "hurry up" does more than a precise timestamp.

---

3. What the Community Feels

Beyond the controlled conditions of academic papers, the lived experience of builders and users on X and in agent communities converges on a single, resonant complaint:

"Everything is now."

Every interaction with a temporally blind agent exists in an eternal, featureless present. Return after a week and the agent has zero sense of discontinuity. No "welcome back." No "I notice it has been a while --- want me to catch you up?" Just the same bright-eyed readiness, as if you had stepped out of the room for exactly zero seconds and the intervening week --- during which you got married, changed jobs, and adopted a cat named Professor Whiskers --- simply did not happen.

Narrative time gaps break hard. The agent treats a three-month absence identically to a three-second pause. It is like calling a friend who has no short-term memory: every conversation starts from scratch, but without the courtesy of admitting it.

There is, however, a complicating nuance that prevents us from simply bolting temporal awareness onto everything and calling it solved:

Sometimes people *want* time blindness.

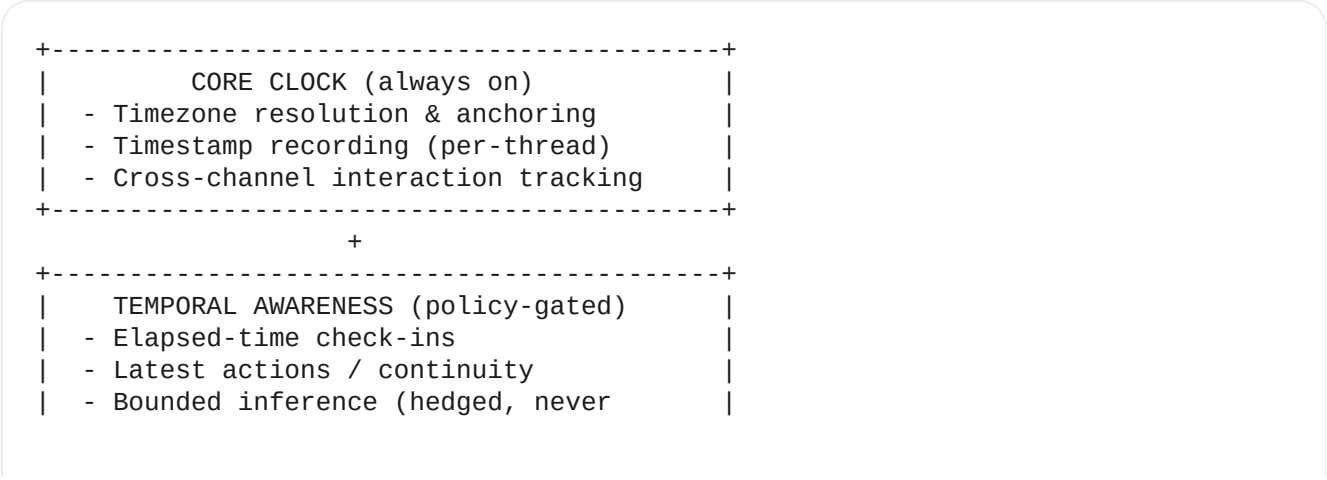
Not every interaction benefits from the agent acknowledging the passage of time. A quick factual lookup does not need to open with "It has been three days since we last spoke, and I want you to know I noticed." That would be exhausting. That would be the conversational equivalent of a retail worker who insists on making eye contact and asking about your weekend when you just want to buy batteries.

This means temporal awareness needs to be **user-configurable** --- a policy gate, not a firehose. Time should surface when it is useful and vanish when it is not. The challenge, as with most things in engineering and in life, is knowing the difference.

---

4. Where CLOCK.md Lands

CLOCK.md is a spec that addresses temporal blindness through two distinct layers, stacked like geological strata but considerably more useful:



```
|    stated as fact)                          |
+---------------------------------------------+
```

The bottom layer --- the Core Clock --- is structural plumbing. It runs all the time, quietly, like the clock on your wall that you never look at but would miss terribly if it stopped. It resolves timezones, records timestamps per thread, and tracks interactions across channels. This is the "making sure the agent knows what day it is" layer, which sounds trivially easy until you remember that most agents currently do not.

The upper layer --- Temporal Awareness --- is where the interesting policy decisions live. This layer is gated, meaning it activates based on configurable rules rather than spraying temporal context into every interaction like an over-enthusiastic perfume salesperson. It handles elapsed-time check-ins, maintains a rolling log of latest actions for continuity, and permits bounded inference --- the kind that is always hedged, always confirmable, and never mistaken for established fact.

What CLOCK.md solves well:

Failure Mode 1 (state missing) gets **full coverage**. The agent now knows when it is, where it is, how long it has been since it last spoke to the user, and in which timezone the user exists. Failure Mode 4 (human expectation misalignment) gets **strong coverage** through policy-gated check-ins whose aggressiveness is configurable --- from gently noting "it has been a while" to proactively offering recaps.

What CLOCK.md does not fully solve (yet):

Failure Mode 2 (state present but unused) still requires a **non-LLM temporal controller** for tool freshness --- because, as the research demonstrated, the model will cheerfully ignore timestamps no matter how prominently you display them. Failure Mode 3 (continuous time / deadlines) requires **per-turn deadline injection**, which lives outside the scope of a context spec.

This is by design. CLOCK.md handles the *context and policy* layer. The remaining gaps need **deterministic middleware** --- which is what the next section is about.

---

5. The Practical Playbook

Eight concrete steps to drag your agents out of the eternal present and into something resembling a functional relationship with time. Each step maps to a specific failure mode, a specific piece of research, or both. None of them involve prayer.

Step 1: Treat time as state, not text

The single most common mistake in temporal agent design is burying time information in prose, as though it were a narrative detail rather than a critical system variable. "It is currently 12:15 PM in Tokyo" is a sentence. What you need is a data structure.

Pass time as explicit, structured fields. Every decision cycle should receive something like the following:

```
temporal_state:
  now_iso: "2026-02-10T12:15:00+09:00"
  session_tz: "Asia/Tokyo"
  agent_tz: "Europe/Berlin"
  elapsed_since_last_interaction: "PT4H32M"
  last_interaction_iso: "2026-02-10T07:43:00+09:00"
  active_deadlines: []
  last_tool_calls:
    web_search: "2026-02-10T12:10:00+09:00"
    calendar_read: "2026-02-10T07:43:00+09:00"
```

This is not poetry. It is not meant to be. It is meant to be parseable, unambiguous, and impossible to misinterpret --- three qualities that natural language has been failing to deliver for approximately six thousand years.

Step 2: Add policy-gated relevance rules

Not every message warrants temporal awareness. A user asking "what is the capital of France" does not need to be greeted with an elapsed-time report and a recap of their last fourteen interactions. That way lies madness, and also very long response times.

Use a policy block --- like CLOCK.md's Section 0 --- to define *when* time matters:

```
checkins:
  rules:
    - when: { thread_kind: project, elapsed_gte: P1D }
      then: { ask_for_updates: true }
    - when: { elapsed_gte: P30D }
      then: { offer_recap: true }
```

The golden rule, if you will permit a single golden rule in a guide that has been resisting the temptation to declare things golden: **make time visible when it matters, and invisible when it does not.** The difference between a helpful temporal agent and an insufferable one is knowing when to shut up about the date.

Step 3: Build a deterministic tool-freshness layer

This directly addresses Failure Mode 2, the one where the agent has timestamps right in front of it and ignores them like a teenager ignoring a pile of laundry. The LLM should not be deciding when to re-call a tool. A deterministic controller should.

Think of it as a caching layer with opinions:

```
tool_freshness:
  classes:
    hot:    { ttl: PT30S, examples: [stock_price, live_score] }
    warm:   { ttl: PT5M,  examples: [weather, news_headlines] }
    cool:   { ttl: PT1H,  examples: [calendar, email_inbox] }
    cold:   { ttl: P1D,   examples: [user_profile, preferences] }
  behavior:
    stale: force_refresh_before_use
    fresh: serve_cached_silently
```

Stock prices are hot --- thirty seconds and they are stale. User profiles are cold --- once a day is plenty, unless your user is undergoing some kind of rapid identity crisis. The controller enforces these TTLs mechanically, without consulting the model's feelings on the matter.

Step 4: Inject remaining time per turn

For any task with a deadline, the remaining time budget must be supplied on every decision step. Not once at the beginning. Not "when it seems relevant." Every single turn. The model has the temporal memory of a goldfish with a head injury, and you must compensate accordingly.

```
DEADLINE: 2 minutes 14 seconds remaining (of original 10 minutes)
Priority: wrap up current subtask; skip nice-to-haves.
```

And remember the counterintuitive finding from the research: a **qualitative urgency cue** can outperform raw numbers. "This is getting tight --- focus on essentials" may produce better behavior than "134 seconds remaining." The model, it turns out, responds to tone. Consider supplying both and letting the vibes do their work.

Step 5: Separate tone inference from action inference

There is a meaningful difference between an agent *saying* something time-aware and an agent *doing* something time-aware. The former is generally harmless and often delightful. The latter can ruin someone's afternoon.

Allow time-based *phrasing* freely:

- "Welcome back --- hope dinner went well" (charming, low-risk)
- "It has been a while! Want a quick recap?" (helpful, still low-risk)

But **require explicit confirmation** before time-inferred *actions*:

- Auto-rescheduling a meeting because "they are probably done by now" (catastrophic)
- Sending a follow-up email based on inferred task completion (presumptuous and potentially career-limiting)

The rule is simple: the agent may *talk* about time all it wants. It may not *act* on temporal inferences without a human in the loop. Inference is cheap. Undoing an incorrectly rescheduled board meeting is not.

Step 6: Keep latest actions small, expiring, and user-controlled

The temptation, when building a continuity system, is to log everything forever. This produces a system that remembers with perfect fidelity that the user went to the gym on March 14th, 2024, which is not useful and is arguably a little creepy.

Instead, maintain a rolling list with a default TTL of 24 hours. Two paths for entry:

1. **Explicit:** The user says something like `clock it: heading to the gym` --- a deliberate act of temporal self-reporting.
2. **Auto:** The agent detects temporal intent in natural language ("going to sleep," "brb," "stepping out for a bit") and logs it automatically, subject to the policy gate from Step 2.

All actions expire. Expired actions get no strong continuity claims. The system forgets, gracefully, like a well-mannered dinner guest who does not bring up things you said three cocktails ago.

Step 7: Index cross-channel interactions with clean scope

Modern humans do not live in a single channel. They message on WhatsApp, email on Gmail, voice-chat on Discord, and occasionally communicate through the ancient ritual of speaking words into the air near another human being. An agent that tracks temporal state must decide how to handle this multi-channel reality.

The default should be conservative: WhatsApp-you and Telegram-you are separate identities unless the user explicitly links them. This respects privacy boundaries and prevents the unsettling experience of an agent on Platform B referencing something you said on Platform A, which feels less like helpfulness and more like surveillance.

Optional, user-approved linking allows cross-channel continuity for those who want it. Group chats introduce additional complexity: per-user timezones, UTC storage internally, and user-specific display formatting. None of this is glamorous. All of it is necessary.

Step 8: Evaluate with the right benchmarks

You cannot fix what you cannot measure, and you cannot measure temporal awareness with standard LLM benchmarks, which were designed to test things like "can the model write a sonnet about a penguin" and not "does the model know it has been forty-five minutes since it last checked the weather."

Build two evaluation shapes:

EVAL TYPE	WHAT IT TESTS	INSPIRED BY
Freshness eval	Does the agent re-call stale tools? Avoid redundant calls on fresh data?	TicToc benchmark
Deadline eval	Does the agent respect time budgets? Adjust behavior as time runs out?	Deadline paper

Without these evaluations, you are flying blind about your agent's time blindness. This is meta-blindness --- a condition so recursive it could qualify for its own research paper.

---

6. Tuning Knobs

CLOCK.md does not ship with a single configuration and a note reading "good luck." It exposes several parameters for per-deployment calibration, because the correct temporal behavior for a customer-support bot is wildly different from the correct temporal behavior for a personal assistant, which is different again from whatever temporal behavior is appropriate for the thing you are building that you will not fully explain to anyone.

KNOB	WHAT IT CONTROLS	DEFAULT
`checkins.rules[].elapsed_gte`	How long before the agent asks "any updates?"	`P1D` for projects, `P30D` for general
`latest_actions.max_items`	Size of the rolling action list	`25`

KNOB	WHAT IT CONTROLS	DEFAULT
`auto_clock_temporal_actions`	Whether "heading to X" style actions auto-store	`true`
`tool_freshness` TTL classes	How long before tool data is considered stale	Per-class, from 30 seconds to 1 day
Deadline feedback style	Numeric countdown vs. qualitative urgency prefix	Both recommended
Per-user calibration	Override duration estimates per activity	Bounded and user-confirmed
Group chat defaults	UTC storage with per-user display timezone	On by default

The recommended starting posture is conservative. Light mode. Explicit clock-it only. One-day check-ins for projects. You can always turn it up later, once you have observed how your users actually interact and confirmed that they will not find aggressive temporal awareness charming in the way that a neighbor who tracks your comings and goings is "charming."

---

7. Closing Thoughts

Temporal blindness is not a quirk. It is not a minor inconvenience, a known limitation to be hand-waved away in the documentation, or a problem that will solve itself when the models get smarter. It is a **fundamental gap** in how large language models process the world, confirmed by rigorous research and experienced daily by everyone building on top of them.

The research is unambiguous: this is real, it is measurable, and it is not solvable by prompting alone. You cannot prompt your way to temporal awareness any more than you can remind a submarine to breathe.

The fix is architectural, and it has four parts:

1. **Structured time state** injected into every decision cycle --- not as prose, not as vibes, but as data.
2. **Deterministic middleware** for tool freshness and deadline tracking --- because the model will not do this for itself, no matter how nicely you ask.
3. **Policy-gated awareness** so that time surfaces only when it helps --- because an agent that always talks about time is almost as bad as one that never does.
4. **Bounded inference** that is hedged, confirmable, and never mistaken for established fact --- because an agent that *acts* on temporal guesses is an agent that will eventually reschedule your wedding.

CLOCK.md is one implementation of these principles. It is battle-tested, open, and designed to be tuned rather than believed. It does not claim to solve everything. It claims to solve the parts that a context-layer spec *can* solve, and to clearly delineate the parts that need heavier machinery.

Your agents do not need to become temporal savants. They do not need to develop a rich inner experience of the passage of time, complete with existential dread and a wistful appreciation for autumn. They just need to stop pretending that every moment is the first moment --- which, when you think about it, is not a terribly high bar. And yet.

Built on research from arXiv:2510.23853 and arXiv:2601.13206. CLOCK.md spec: v1.2 --- open and free to use.